React · Guía de repaso

Programación 4 | Parcial 1 · Respuestas a las seis hojas compartidas. Las letras corresponden a las opciones de los originales.

1. Formularios controlados

1. ¿Qué define a un componente controlado?

Opción b. El valor del input está vinculado al estado de React. Se muestra con `value` y se actualiza mediante onChange. [1]

2. ¿Qué ocurre con value vinculado al estado, pero sin onChange?

Opción c. El texto queda de solo lectura: lo escrito no se conserva porque el estado no se actualiza. React avisa en consola; la aplicación no se detiene. [1]

3. ¿Para qué sirve event.preventDefault() en onSubmit?

Opción a. Cancela el envío nativo que recarga o navega la página y permite manejarlo con JavaScript. No limpia ni valida los campos por sí mismo. [1]

Análisis de FormularioUsuario: los dos errores y sus soluciones

Error 1: falta onChange; agregar `onChange={(e) => setEmail(e.target.value)}` para sincronizar el estado.
Error 2: falta `e.preventDefault()` en `manejarEnvio`; llamarlo al comienzo para evitar el envío nativo. [1]

Código corregido

```
import { useState } from 'react';

export default function FormularioUsuario() {
  const [email, setEmail] = useState('');
  const manejarEnvio = (e) => {
    e.preventDefault();
    console.log('Datos enviados:', email);
  };

  return (
    <form onSubmit={manejarEnvio}>
      <label>
        Email:
        <input type="email" value={email}
          onChange={(e) => setEmail(e.target.value)} />
      </label>
      <button type="submit">Enviar</button>
    </form>
  );
}
```

El ejemplo registra el email en consola. Para enviarlo a un servidor haría falta una petición, por ejemplo con fetch.

2. useEffect y consumo de API

Respuestas a la hoja marcada «UE» y a su ejercicio de análisis de código.

1. ¿Cuándo se ejecuta useEffect(() => { fetchData(); }, [])?

Opción b. Al montar el componente, después del render inicial. El arreglo vacío evita repetirlo por cambios de estado durante ese montaje. [2]

2. ¿Cuál es el propósito de isLoading?

Opción b. Indicar si la petición está en curso para mostrar condicionalmente un mensaje de carga o un spinner. No guarda los usuarios ni evita ciclos por sí solo.

3. ¿Qué dispara el efecto con [provincia]?

Opción a. Un cambio en `provincia` vuelve a ejecutarlo. También se ejecuta al montar; no espera al clic de envío. [2]

Análisis 1. ¿Qué pasa al quitar [] del efecto que consulta usuarios?

El efecto se ejecuta después de cada render confirmado. Como actualiza `personas` e `isLoading`, provoca nuevos renders y nuevas peticiones: un ciclo repetitivo que consume recursos y puede saturar la API. [2]

Análisis 2. ¿Qué tipo de dato guarda personas al hacer setPersonas(data.results)?

Un **arreglo de objetos**: cada objeto representa un usuario. En el ejemplo se solicitan 50. Se guarda `data.results`, no el objeto completo de la respuesta.

Comparación rápida de dependencias [2]

Final del efecto	Cuándo se ejecuta
`}, []);`	Al montar. No se repite por cambios de dependencias.
`}, [provincia]);`	Al montar y cuando cambia provincia.
`});`	Después de cada render confirmado.

Precisión: con StrictMode, React puede ejecutar un ciclo extra de configuración y limpieza en desarrollo. Esto no cambia que b sea la opción esperada del parcial para []. [2]

Lectura del flujo del código

isLoading = true → fetch espera la respuesta → response.json() lee el JSON → setPersonas guarda el arreglo → isLoading = false. Si la petición falla, el código original no llega a la última línea; conviene manejar errores con try/catch/finally. [6]

3. Estructura, DOM y estado

Respuestas a las dos hojas del tema «Estado».

1. ¿Cuál es la función principal de main.jsx?

Opción b. Importar App y montar la aplicación en el DOM mediante ReactDOM, normalmente con `createRoot(...).render(<App />)`. [5]

2. ¿Cómo se relacionan index.html, main.jsx y App.jsx al iniciar?

`index.html` contiene el nodo raíz y carga `main.jsx` como módulo. Este importa `App.jsx` y renderiza su componente dentro del nodo raíz. App define la interfaz principal. [5]

3. ¿En qué contenedor de index.html se monta la aplicación?

Opción a. En `<div id="root"></div>`, según la estructura del ejercicio. El script carga el código; no es el contenedor visual.

4. ¿Qué es el Virtual DOM y por qué se utiliza?

Es una representación de la interfaz en memoria. React compara la nueva estructura con la anterior y calcula las operaciones necesarias sobre el DOM real, sin reconstruir toda la página en cada cambio. [4]

5. ¿Cómo se llama el proceso de comparación y actualización?

Opción b. Reconciliación. El diffing es la comparación de las estructuras para detectar diferencias dentro de ese proceso. [4]

6. Primer clic en «Incrementar»: valores exactos

Inciso	Resultado	Motivo
A · Normal en pantalla	**0**	El nuevo render ejecuta otra vez let contadorNormal = 0.
B · Estado en pantalla	**1**	El setter conserva el nuevo estado para el siguiente render.
C · Consola	`Normal: 1`	El log lee la variable incrementada dentro del manejador actual.

7. ¿Por qué contadorNormal no actualiza la pantalla y contadorEstado sí?

Modificar una variable local no solicita un render y su valor se reinicia al ejecutar de nuevo el componente. `useState` conserva el valor entre renders, y su setter solicita la actualización de la interfaz. En este código, el setter del estado causa el render que vuelve a poner el contador normal en 0. [3]

4. Preguntas similares

Práctica propuesta a partir de estos temas; no son preguntas confirmadas del examen.

1. ¿En qué se diferencian value y defaultValue?

`value` controla el valor actual; `defaultValue` solo establece el inicial de un input no controlado. [1]

2. ¿Cómo se vacía un input controlado después de enviar?

Actualizando su estado: `setEmail('')`. El input reflejará ese valor vacío. [1]

3. ¿Cómo se controla un checkbox?

Con `checked={marcado}` y `onChange={(e) => setMarcado(e.target.checked)}`. [1]

4. ¿Por qué conviene manejar el envío con onSubmit del form?

Centraliza el envío, tanto al usar el botón submit como al enviar con Enter en un campo que lo permita. Allí se cancela el comportamiento nativo. [1]

5. ¿Qué sucede si el efecto de localidades usa [] en vez de [provincia]?

No vuelve a consultar por un cambio de provincia; solo ejecuta la consulta inicial durante ese montaje. [2]

6. ¿Para qué sirve la función de limpieza de un efecto?

Para liberar recursos o cancelar trabajo previo. Se ejecuta antes de repetir el efecto por dependencias cambiadas y al desmontar. [2]

7. ¿Para qué sirven await fetch(...) y await response.json()?

El primero espera la respuesta HTTP; el segundo espera que se lea y convierta su cuerpo JSON en un valor de JavaScript. [6]

8. ¿Cómo evitar que el indicador de carga quede activo si falla la petición?

Usar `try/catch` para gestionar el error y `finally` con `setIsLoading(false)`. Además, revisar `response.ok`: fetch no rechaza automáticamente por un 404 o 500. [6]

9. ¿Qué mostraría el contador original después de tres clics separados?

Pantalla: **Normal: 0** y **Estado: 3**. Consola: `Normal: 1` en cada clic, porque la variable local se reinicia en cada render. [3]

10. ¿Cómo incrementar usando el valor de estado anterior?

`setContadorEstado(prev => prev + 1)`. Es útil al encadenar actualizaciones que dependen del valor anterior.

11. ¿Un nuevo render significa reemplazar todo el DOM?

No. React calcula la nueva interfaz y modifica los nodos del DOM real que requieren cambios. [4]

12. ¿Qué ocurre si main.jsx busca un id root que no existe?

Obtiene `null`; createRoot necesita un elemento DOM válido y no puede montar la aplicación allí. [5]

Referencias de verificación

[1] React: inputs y formularios

[2] React: useEffect

[3] React: memoria del componente

[4] React: render y commit

[5] React: createRoot

[6] MDN: uso de Fetch